

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:		Reg. No.:	
Date:		Duration:	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – CI/CD Pipeline Design Exercise

Instructions to Students:

Each student is assigned an individual problem statement. Based on the assigned problem statement, **design a CI/CD (Continuous Integration and Continuous Deployment) pipeline** for the proposed software system to automate the processes of code integration, building, testing, and deployment.

The exercise should include:

1. **Analyze the project requirements** and identify the CI/CD needs of the assigned problem statement.
2. **Design the CI/CD pipeline workflow** showing stages such as source code management, build, testing, code quality analysis, packaging, and deployment.
3. Identify suitable **tools and technologies** for each stage of the pipeline and justify their selection.
4. Define appropriate **automated testing strategies** to be integrated into the pipeline.
5. Design the **deployment strategy**, including development, testing/staging, and production environments.
6. Incorporate appropriate **security, quality checks, notifications, and rollback mechanisms** in the pipeline.
7. Prepare a **CI/CD pipeline diagram** and explain the workflow from code commit to deployment.

Deliverable: Submit a **CI/CD Pipeline Design document** containing the pipeline architecture/diagram, stages, tools, configuration/workflow, testing strategy, deployment process, and justification based on the assigned problem statement.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Pipeline Analysis(15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear or incomplete
Pipeline Architecture & Workflow(25%)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or poorly structured
Tools & Technology Selection(20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are identified but justification is limited	Tools are inappropriate or not clearly identified
Automated Testing & Quality Integration(15%)	Effectively integrates automated testing and code-quality/security checks into the pipeline	Includes major testing and quality checks	Includes basic testing with limited integration	Testing/quality checks are missing or inappropriate
Deployment, Security & Rollback Strategy(15%)	Clearly designs deployment environments, security practices, monitoring, and rollback strategy	Covers deployment and most security/rollback aspects	Basic deployment strategy with limited security/rollback details	Deployment strategy is incomplete or lacks security considerations
Pipeline Diagram & Documentation (10%)	Clear, accurate, professional diagram with excellent explanation and documentation	Well-presented diagram and documentation with minor issues	Adequate diagram/documentation but lacks clarity	Poorly presented, incomplete, or difficult to understand

Criteria	Mark
Requirement & Pipeline Analysis(15%)	/5
Pipeline Architecture & Workflow(25%)	/4
Tools & Technology Selection(20%)	/4
Automated Testing & Quality Integration(15%)	/4

Deployment, Security & Rollback Strategy(15%)	/4
Pipeline Diagram & Documentation(10%)	/4
TOTAL	/25

Problem Statement : AI-Powered Digital Twin Platform for Smart City Infrastructure

Industry Context

Smart cities increasingly adopt digital twin technology to simulate and monitor urban infrastructure in real time. AI, IoT, GIS, and cloud computing enable virtual replicas of roads, utilities, buildings, and transportation systems for predictive maintenance and strategic planning.

Problem Statement

Urban infrastructure management is challenging due to aging assets, increasing population, and limited real-time visibility. Traditional monitoring systems provide fragmented information and lack predictive capabilities. An AI-powered digital twin platform should create virtual models of city infrastructure by integrating real-time IoT data. The system should simulate infrastructure performance, detect anomalies, predict failures, and support data-driven urban planning. This solution will improve infrastructure reliability and maintenance efficiency.

Objectives

1. Create digital twins of city infrastructure.
2. Integrate real-time IoT data.
3. Predict infrastructure failures.
4. Monitor asset health continuously.
5. Support predictive maintenance.
6. Simulate urban scenarios.
7. Generate infrastructure performance reports.
8. Improve smart city planning.

Expected Outcomes

1. Enhanced infrastructure monitoring.
2. Reduced maintenance costs.
3. Improved asset reliability.
4. Better urban planning decisions.
5. Accurate predictive maintenance.
6. Increased operational efficiency.
7. Real-time infrastructure visualization.
8. Sustainable city development.

INTRODUCTION

1. Introduction

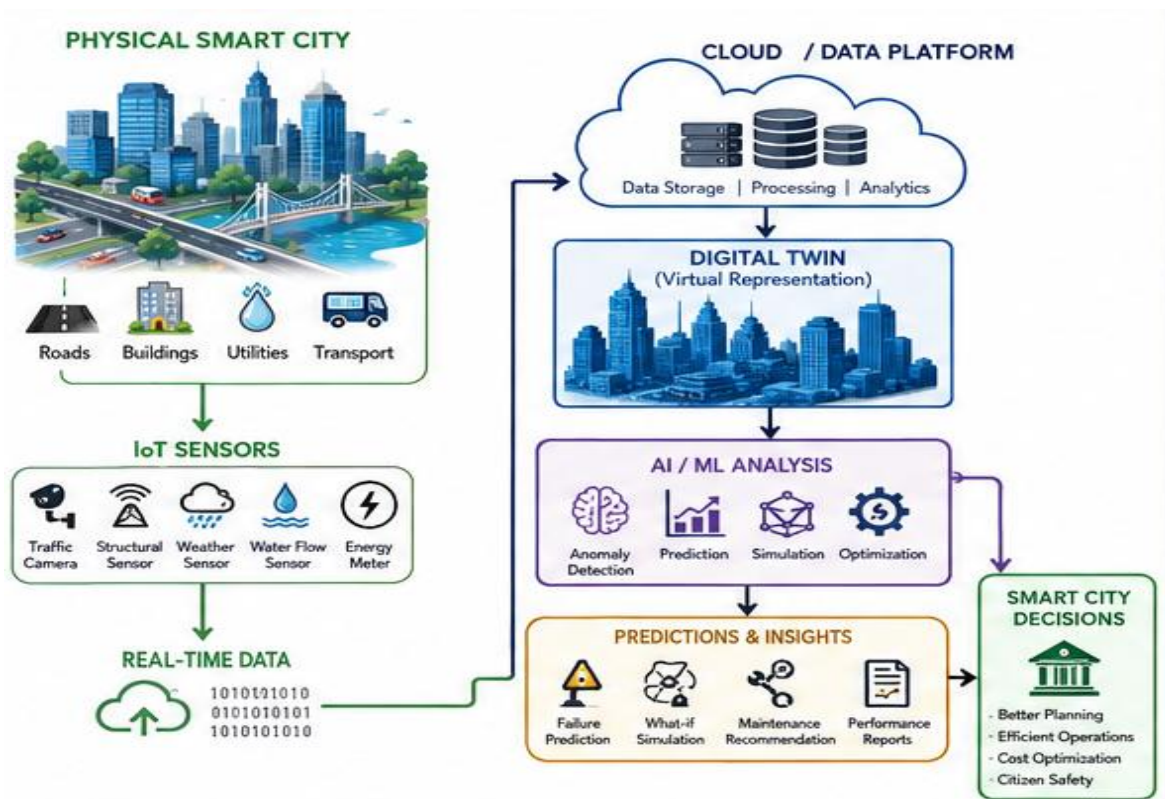
Smart cities use IoT, AI, GIS and cloud computing to monitor and manage urban infrastructure. A **Digital Twin** creates a virtual representation of physical assets such as roads, buildings, utilities and transportation systems.

The proposed platform collects real-time IoT data and uses AI to monitor asset health, detect anomalies, predict failures and support infrastructure planning.

A CI/CD pipeline is required to continuously integrate, test, secure and deploy the platform components reliably.

Objectives

- Create digital twins of city infrastructure.
- Integrate real-time IoT data.
- Detect anomalies and predict failures.
- Continuously monitor infrastructure health.
- Support predictive maintenance.
- Simulate urban scenarios.
- Generate infrastructure reports.
- Improve smart city planning.



PROBLEM ANALYSIS

2. Problem Statement

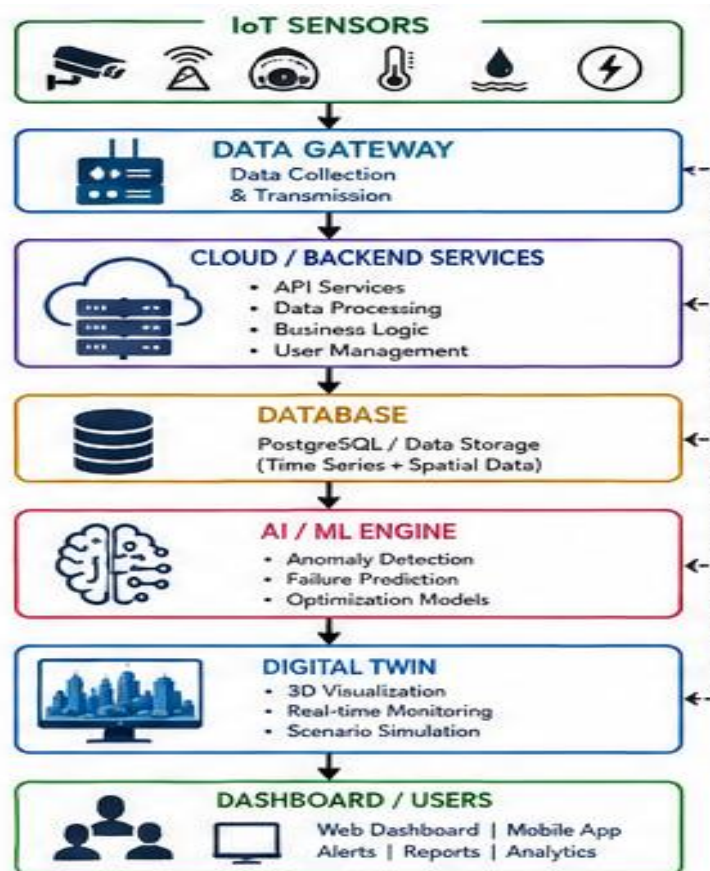
Urban infrastructure faces aging assets, increasing population and limited real-time visibility. Traditional monitoring systems often provide fragmented information and lack predictive capabilities.

The proposed digital twin platform integrates infrastructure data into a unified system and uses AI for prediction and decision support.

CI/CD Requirements

The pipeline should provide:

1. Version-controlled source code.
2. Automated application build.
3. Automated testing.
4. Code-quality analysis.
5. Security checks.
6. Containerized packaging.
7. Development, staging and production deployment.



CI/CD PIPELINE ARCHITECTURE

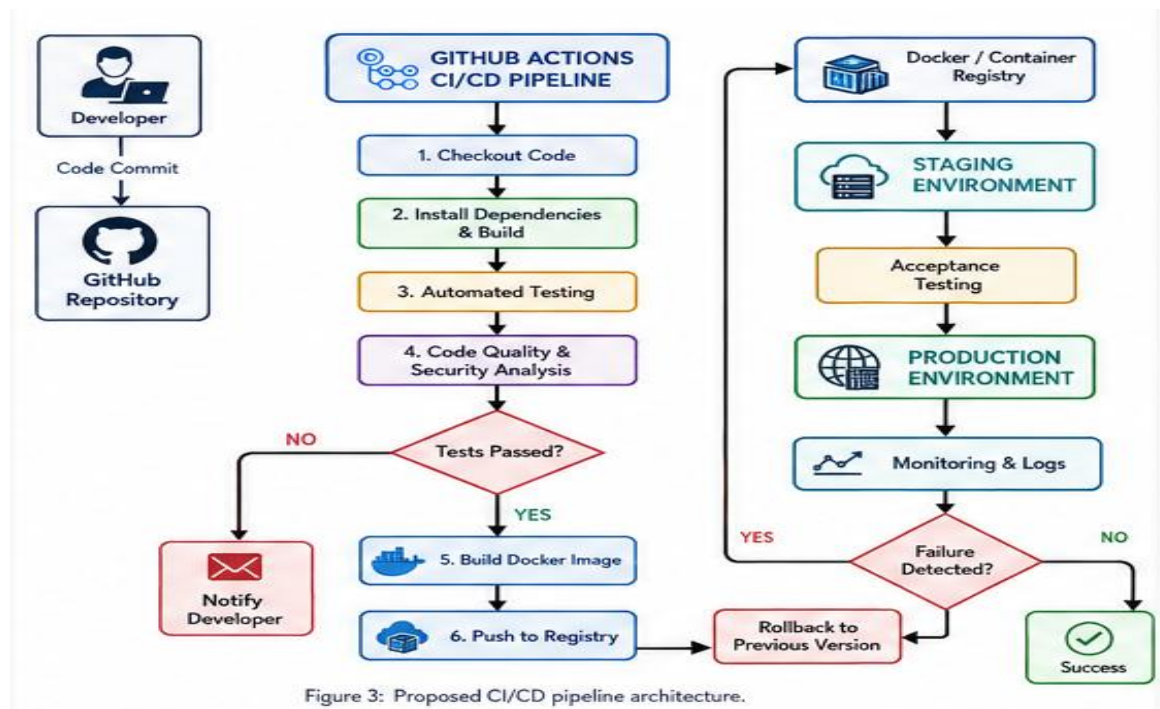
3. Proposed CI/CD Architecture

The proposed pipeline automates the complete software delivery process from code commit to production deployment.

Workflow

Developer → GitHub → GitHub Actions → Build → Test → Quality & Security → Docker → Registry → Staging → Acceptance Testing → Production → Monitoring

If testing fails, the pipeline stops and the developer is notified. If deployment fails, the previous stable version is restored.



The pipeline covers source control, build, testing, quality analysis, packaging and deployment, as required by the assessment rubric.

PIPELINE STAGES

4. CI/CD Pipeline Stages

4.1 Source Code Management

Git and GitHub are used to maintain and manage application source code. Developers work using feature branches and pull requests.

4.2 Build

The pipeline installs dependencies and builds the frontend, backend and AI services.

4.3 Automated Testing

Unit, integration, API and AI model tests are executed automatically.

4.4 Quality & Security

Source code is analyzed for bugs, vulnerabilities and code-quality issues.

4.5 Packaging

Docker containers are created for application services.

4.6 Deployment

The validated application is deployed to staging and then production.

4.7 Monitoring

The deployed system is continuously monitored using application and infrastructure metrics.

TOOLS AND TECHNOLOGIES

5. Tools & Technology Selection

Stage	Tool	Purpose
Source Control	Git + GitHub	Version management
CI/CD	GitHub Actions	Pipeline automation
Frontend	React	Digital twin dashboard
Backend	Node.js / Python	APIs and services
AI/ML	Python	Prediction and anomaly detection
Database	PostgreSQL	Infrastructure data
Containerization	Docker	Application packaging
Quality	SonarQube	Code-quality analysis
Testing	Jest / PyTest	Automated testing
API Testing	Postman	API validation
Registry	Docker Registry	Container storage
Monitoring	Prometheus + Grafana	Metrics and dashboards

Justification

GitHub provides source control and collaboration, GitHub Actions automates the pipeline, Docker provides consistent deployment environments, SonarQube checks code quality, and Prometheus/Grafana support continuous monitoring.

The selected tools are mapped to the individual pipeline stages as expected by the assessment rubric.

AUTOMATED TESTING

6. Automated Testing & Quality

Testing is integrated into the CI pipeline to identify defects before deployment.

Testing Types

- **Unit Testing:** Tests individual functions and components.
- **Integration Testing:** Verifies communication between services.
- **API Testing:** Validates API requests and responses.
- **AI Model Testing:** Checks prediction performance using accuracy, precision, recall and F1-score.
- **Security Testing:** Detects vulnerable dependencies and security issues.

Quality Gate

Build

↓

Unit Test

↓

Integration Test

↓

API Test

↓

AI Model Test

↓

Security Scan

↓

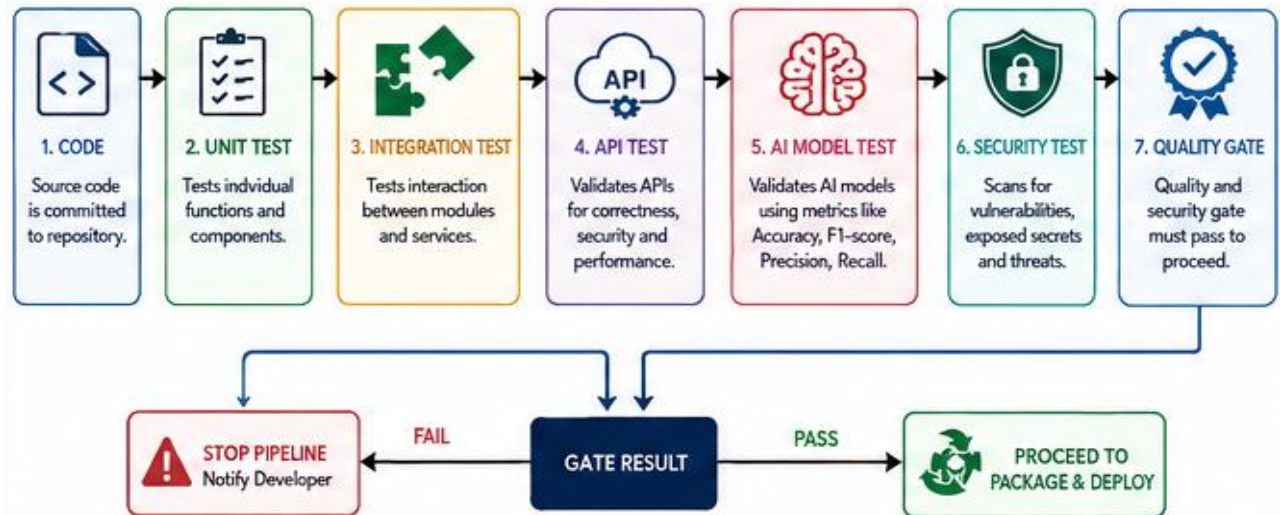
Quality Gate

↓

PASS → Package & Deploy

FAIL → Stop Pipeline

Automated testing and quality checks ensure that only validated software reaches deployment. This directly addresses the assessment's automated testing and quality-integration criteria.



DEPLOYMENT STRATEGY

7. Deployment Environments

The system uses three environments.

Development

Used for developing and testing new features.

Staging

Used for integration, performance, security and acceptance testing before production.

Production

Hosts the live digital twin platform and provides real-time infrastructure monitoring.

Deployment Flow

Development → Staging → Testing → Production → Monitoring

Blue-Green Deployment

A Blue-Green strategy can be used to reduce deployment risk. The new version is deployed separately and validated before receiving user traffic.

If the new version fails, traffic can be redirected to the previous stable version.

SECURITY, MONITORING & ROLLBACK

8. Security Strategy

Security is incorporated throughout the pipeline.

- Store API keys and passwords using secure secrets.
- Scan dependencies for vulnerabilities.
- Scan Docker images before deployment.
- Apply role-based access control.

8.1 Monitoring

Prometheus collects system and application metrics, while Grafana provides monitoring dashboards.

Important metrics include:

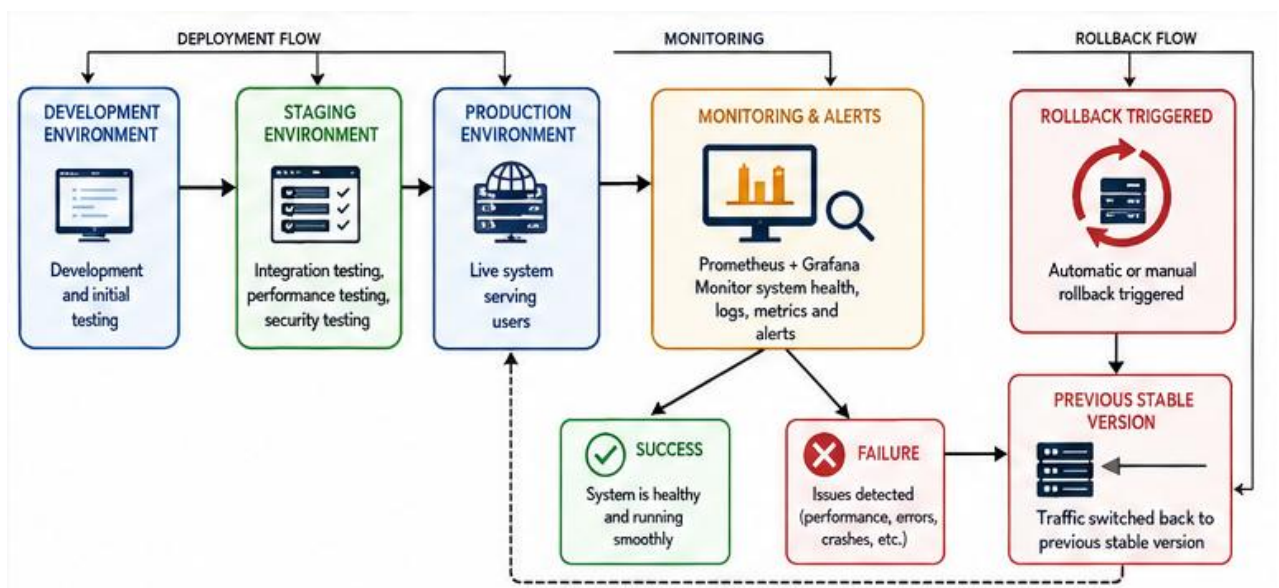
- CPU and memory usage
- API response time
- Application errors
- Container health

8.2 Rollback Strategy

If a production deployment fails, the pipeline restores the previous stable version.

Failure → Health Check → Rollback → Previous Stable Version

Rollback may be triggered by application crashes, failed health checks, high error rates or severe performance problems.



COMPLETE WORKFLOW

9. End-to-End CI/CD Workflow

1. Developer creates or modifies a feature.
2. Code is pushed to GitHub.
3. GitHub Actions starts the pipeline.
4. Dependencies are installed and the application is built.
5. Automated tests are executed.
6. Code quality and security checks are performed.
7. Docker images are created.
8. Images are stored in the container registry.
9. The application is deployed to staging.
10. Acceptance testing is performed.
11. The approved version is deployed to production.
12. Prometheus and Grafana monitor the system.
13. Failed releases are rolled back.

Pipeline Decision

Test/Quality PASS → Deployment

Test/Quality FAIL → Stop Pipeline + Notification

Production Failure → Rollback

The complete workflow demonstrates the movement from code commit through testing and deployment, as required in the assessment.

BENEFITS & CONCLUSION

10. Benefits

- Faster software delivery.
- Improved application quality.
- Automated testing and validation.
- Better security.
- Reduced deployment risk.
- Quick failure recovery.

- Continuous system monitoring.
- Consistent deployment through containers.
- Improved infrastructure management.

11. Expected Outcomes

1. Enhanced infrastructure monitoring.
2. Reduced maintenance costs.
3. Improved asset reliability.
4. Better urban planning.
5. Predictive maintenance.
6. Increased operational efficiency.
7. Real-time infrastructure visualization.
8. Sustainable city development.

12. Conclusion

The proposed CI/CD pipeline provides a reliable approach for developing, testing, securing and deploying the AI-powered digital twin platform. Automated testing, quality checks, containerization, staged deployment, monitoring and rollback improve the reliability and maintainability of the system.

The pipeline therefore supports continuous delivery of a secure and dependable smart city infrastructure platform.